Sending Messages from MQL5 to Discord, Creating a Discord-MetaTrader 5 Bot

Omega J Msigwa | 26 June, 2025

Contents

- Introduction
- Creating a Discord Webhook
- Sending the first message from MetaTrader 5 to Discord
- Creating a Discord class in MQL5
- Discord bot's identity
- Working with embeds and image files
- Sending trade notifications to Discord
- Conclusion

Introduction

We are no longer living in the early stages of the internet and the primal digital era; nowadays, almost everything can be connected with anything on the internet; it only depends on one's willingness to do the work required.

The presence of API's (Application Programming Interface) which is a set of rules and protocols that allows different software applications to communicate with each other, have made it easier for more than one software or application to connect to another and hence it has made the most connected internet that we see today.

To use any API provided, you have to adhere to the rules and protocols and not to mention you must be granted a secure way *(if there is any)* to access it by the API provider.

Making a communication between MetaTrader 5 and external applications is not a new thing; it has been done before on several applications that offer reliable API(s) that MQL5 developers use to send and receive information using Web Requests. The most common app that MQL5 traders use for such communications is Telegram.

In this article, we are going to discuss how we can send messages and trading information (signals) from MetaTrader 5 to Discord, using the MQL5 programming language.

Creating a Discord Webhook

For those who aren't yet familiar with Discord:

> Discord is a free communication platform that allows users to chat via text, voice, and video, and share screens. It's popular for connecting with friends and communities, particularly within online gaming, but is also used for various other groups like book clubs or sewing circles.

Similarly to Telegram, this is a communication platform that allows users to communicate with each other. Both these platforms provide API's which allow communications even outside their platforms, but *Discord if far ahead of telegram when it comes to the integration, automation aspects*.

This platform allows users to create flexible communities and manage apps (also known as bots) of different kinds.

There are various ways you can create bots and automate communications and the process of sharing information to Discord, but the simplest way is using a *Webhook* in one of the channels from your community. Here is how you create one.

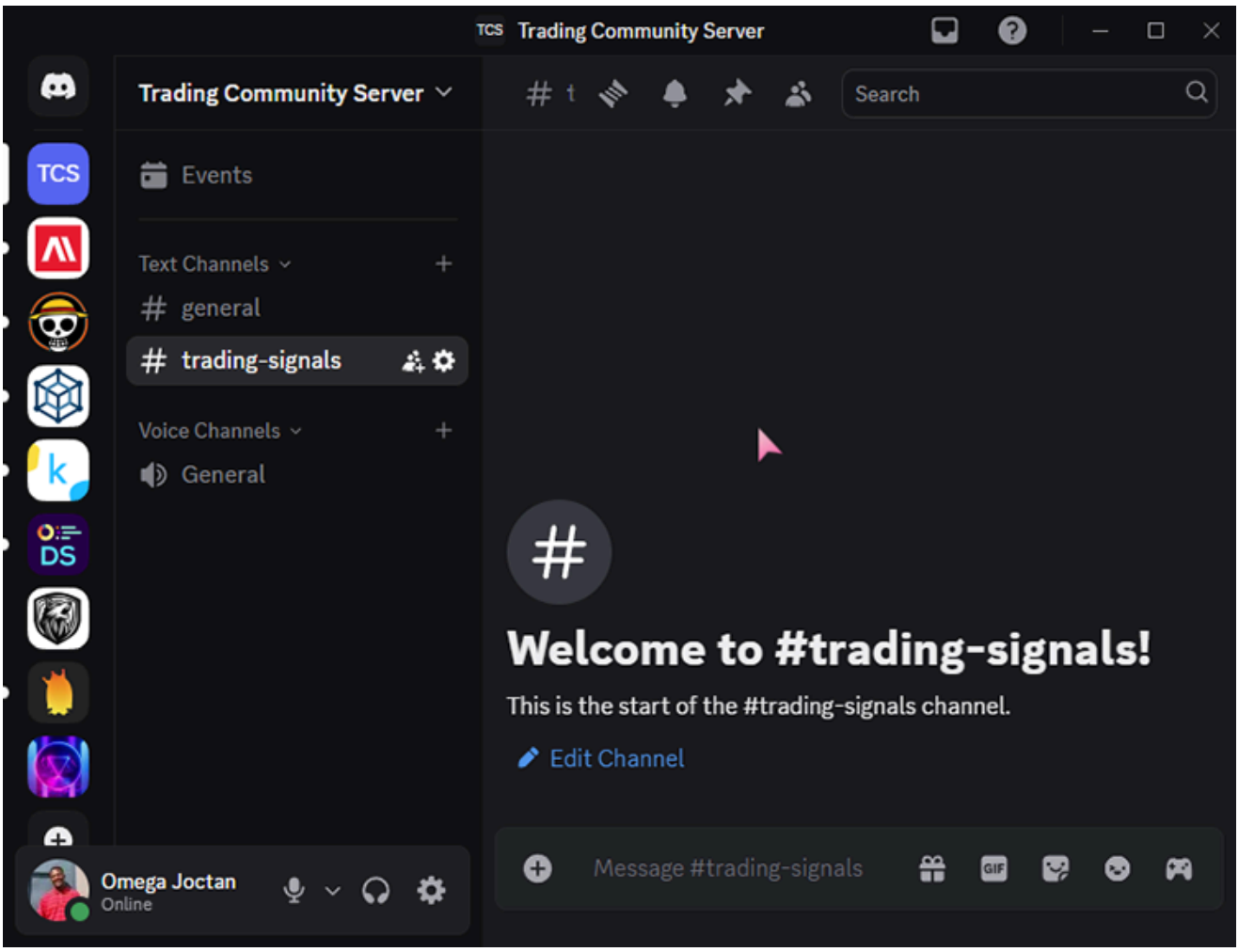


Figure 0.

From your community, go to *server settings*.

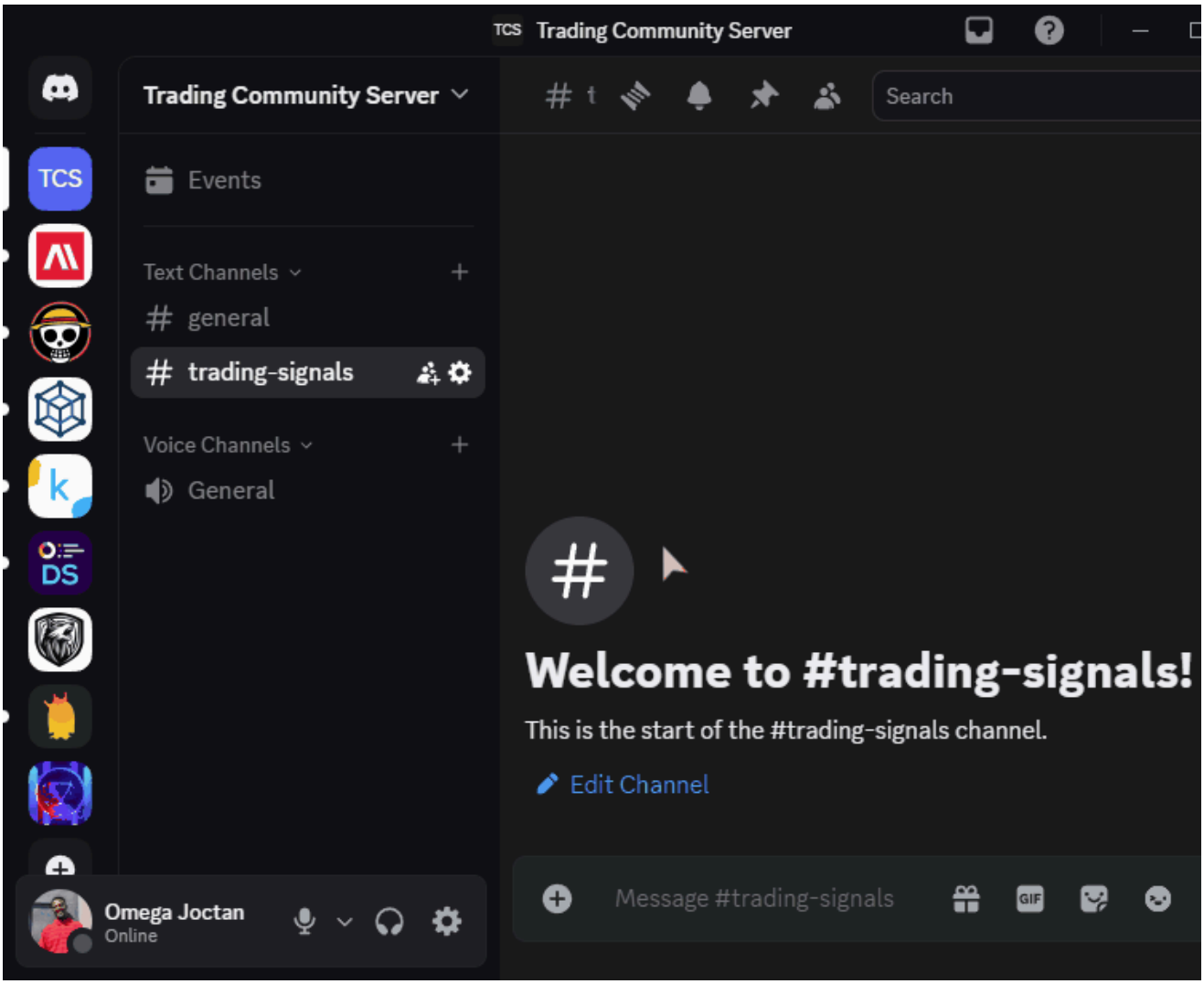


Figure 02.

From **server settings** go to *Integrations* > *Webhooks*.

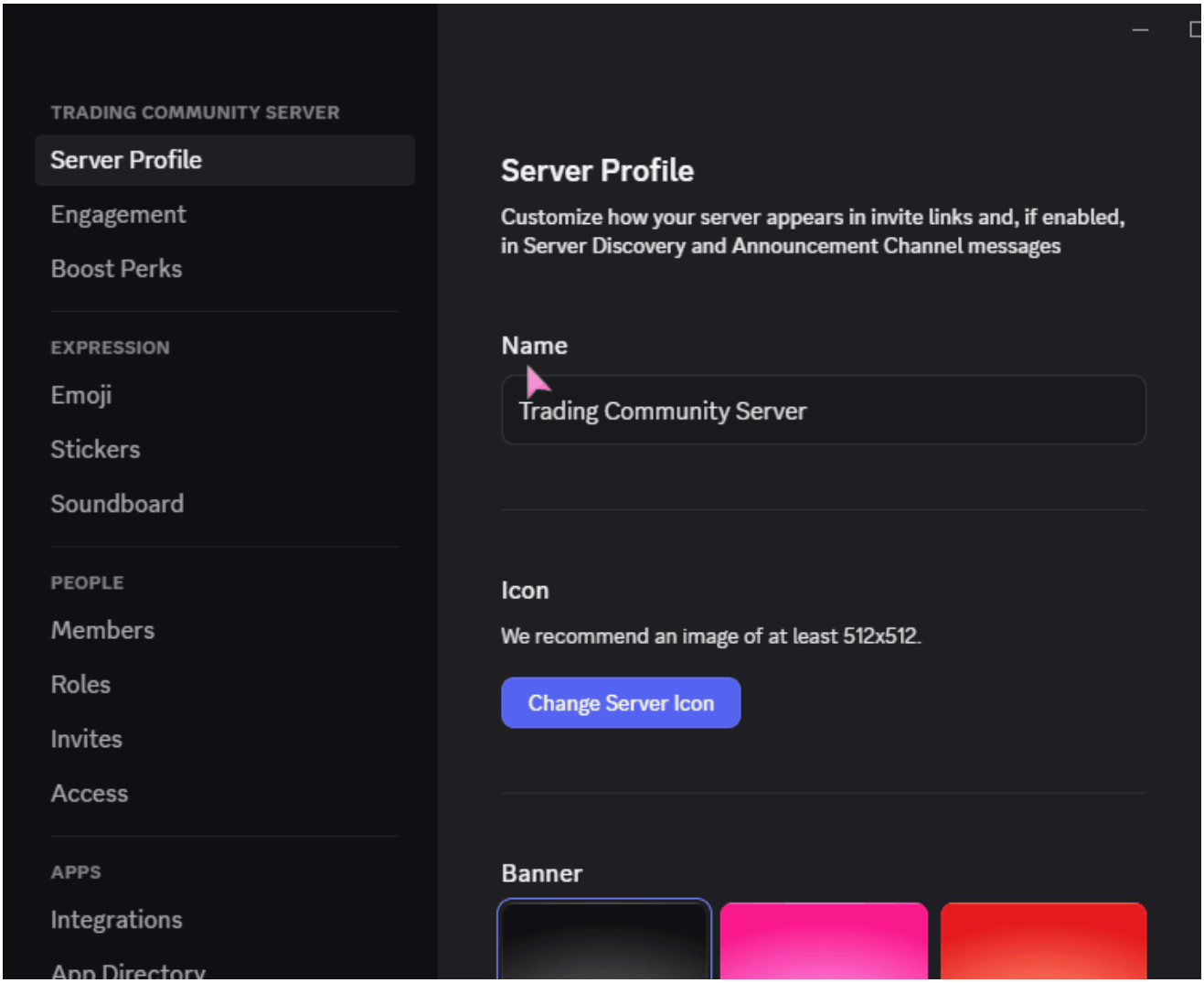


Figure 03.

Click the button *New Webhook* to create a Webhook. *A Webhook with the default name Captain Hook will be created*. Feel free to modify this name to whatever name you want.

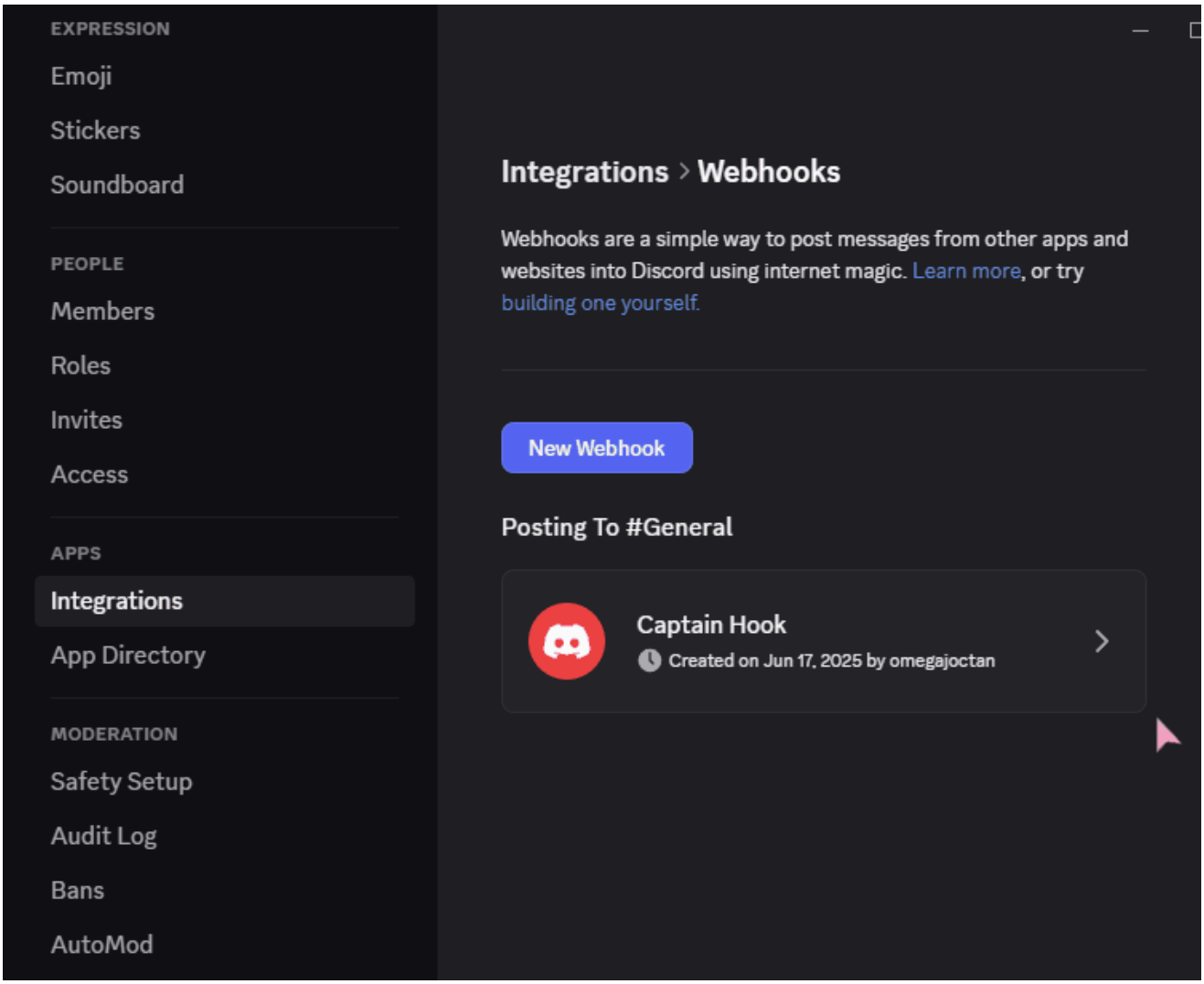


Figure 04.

After modifying its name, you can select the channel where you want to apply this webhook to. In this server, I have a channel named trading signals, which is where I want to apply this Webhook to. *See figure 01*.

Finally, we have to copy *the Webhook URL*. This *is our API gateway* that we can *use with* [*Web Requests in MQL5*](). **.**

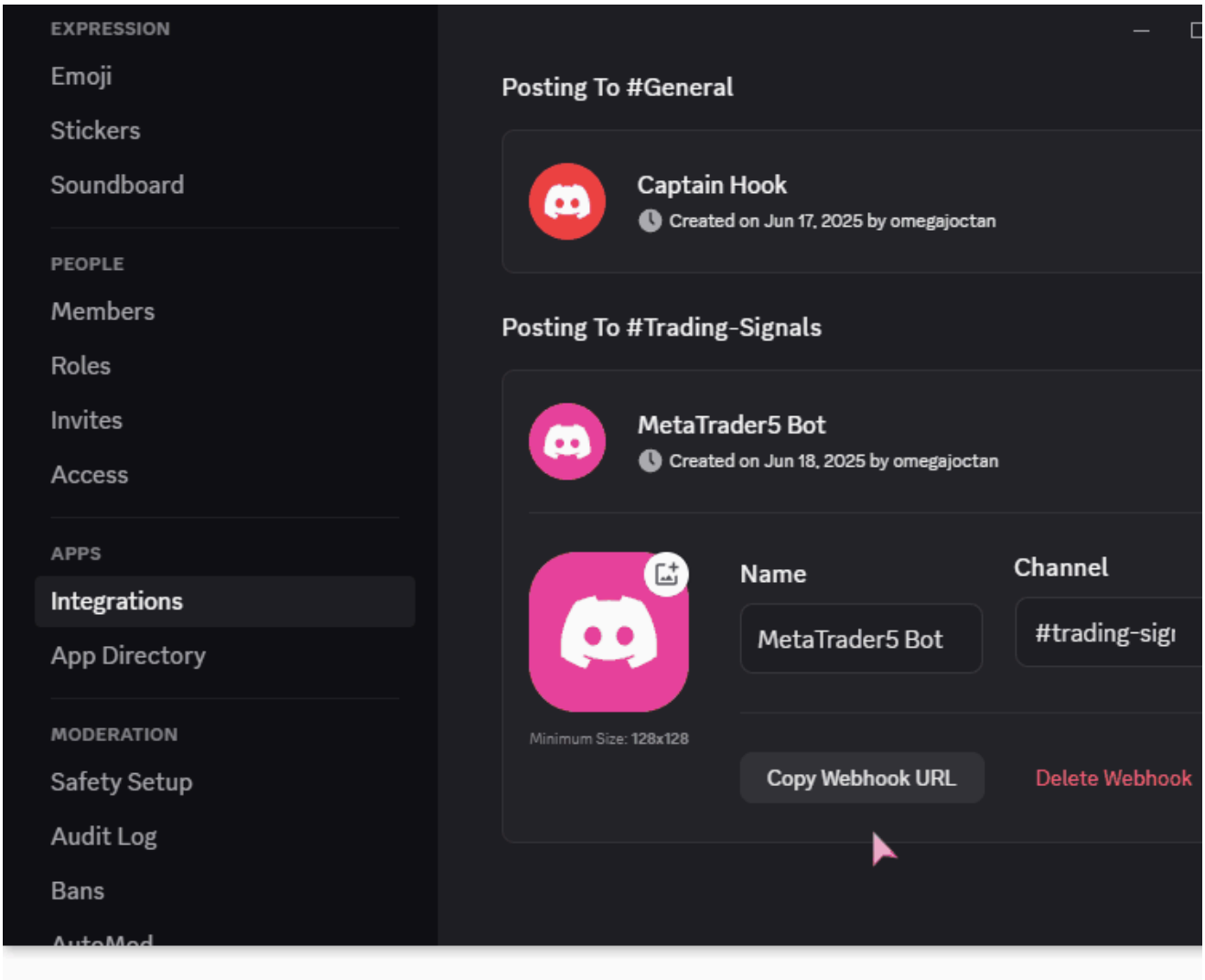
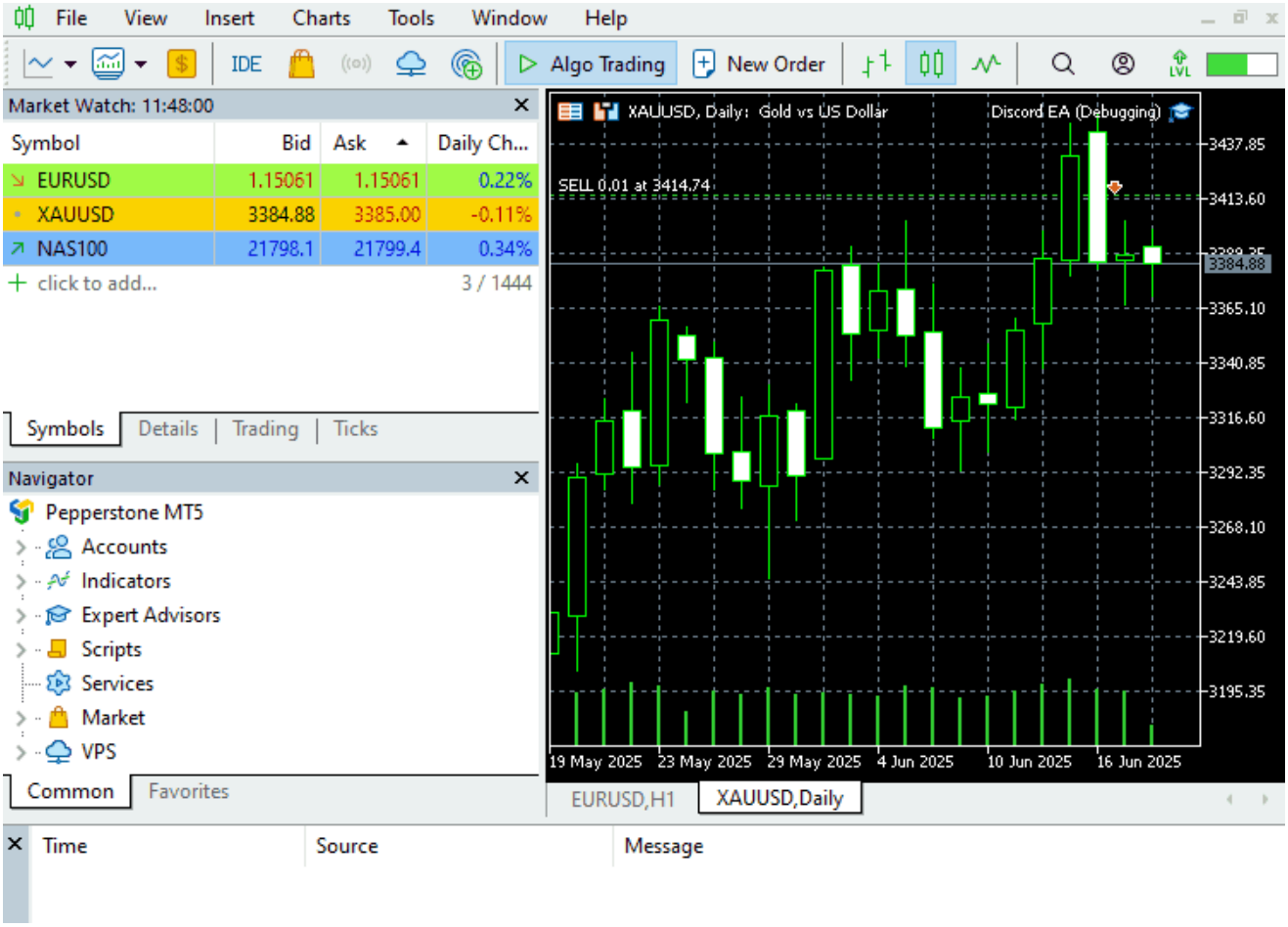


Figure 05.

Sending the First Message from MetaTrader 5 to Discord

The first thing we have to do is add https://discord.com to the list of allowed URLs in MetaTrader 5 otherwise, everything we are going to discuss afterward won't work.

In MetaTrader 5 go to *Tools > Options>Expert Advisors > Ensure Allow WebRequest for listed URL is checked*, then proceed to *add a URL on the form below.*



Using the Webhook URL, we can send a POST request to this API endpoint given a JSON-formatted data.

Filename: Discord EA.mq5

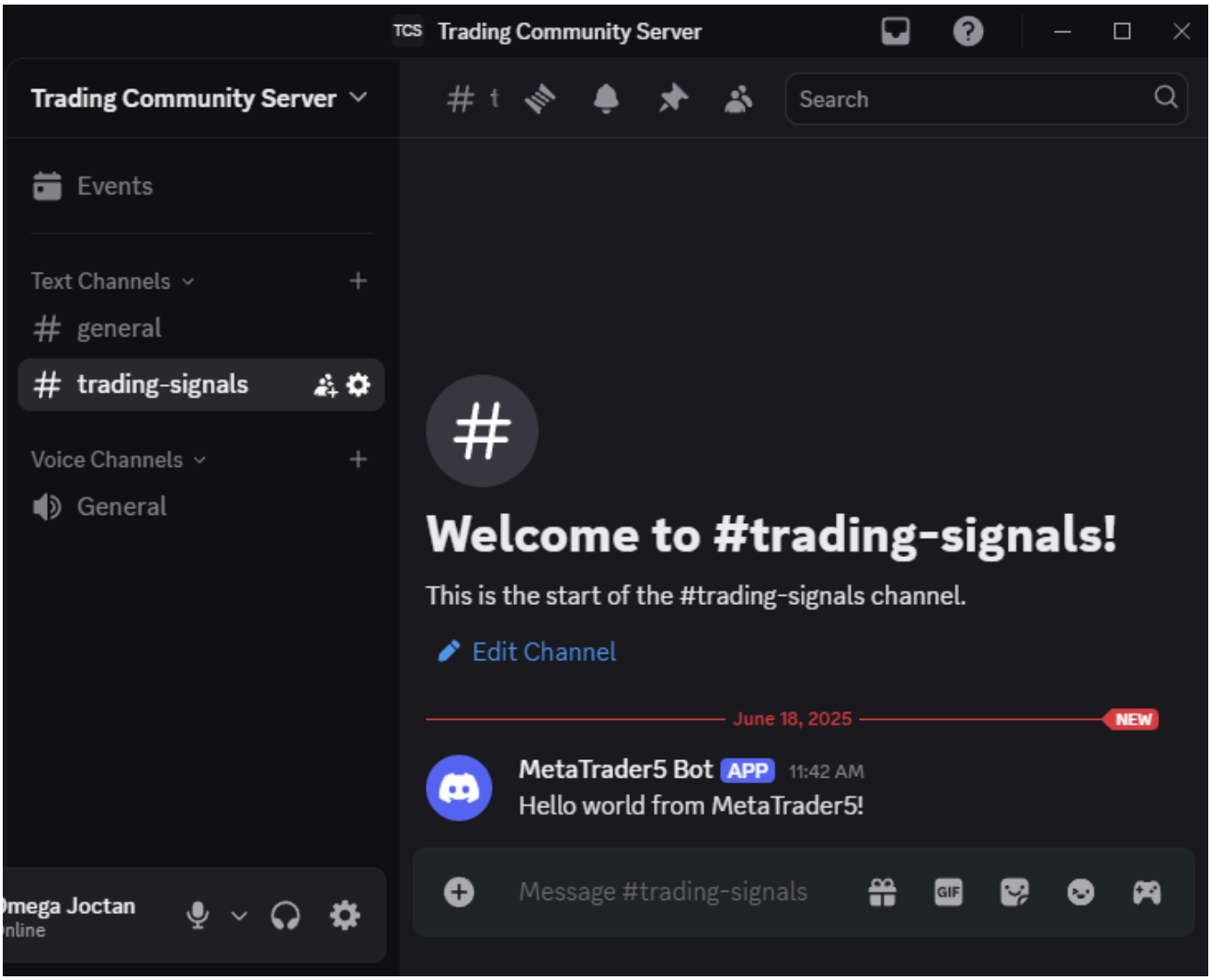
```
#include <jason.mqh> //https://www.mql5.com/en/code/13663

string discord_webhook_url = "https://discord.com/api/webhooks/1384809399767269527/105Kp2
//+------------------------------------------------------------------+
//| Expert initialization function                                   |
//+------------------------------------------------------------------+
int OnInit()
  {
//---

   CJAVal js(NULL, jtUNDEF); //Creating a json object

   string raw_json = "{\"content\": \"Hello world from MetaTrader5!\"}";

   bool b = js.Deserialize(raw_json); //Parse JSON string into a structured object

   string json;
   js.Serialize(json); //Convert the object to a valid JSON string

//--- Sending a Post webrequest

   char data[];
   ArrayResize(data, StringToCharArray(json, data, 0, StringLen(json))); //--- serialize

//--- send data

   char res_data[]; //For storing the body of the response
   string res_headers=NULL; //For storing response headers (if needed)

   string headers="Content-Type: application/json; charset=utf-8";
   uint timeout=10000; //10 seconds timeout for the HTTP request

   int ret = WebRequest("POST",
                        discord_webhook_url,
```

```
                            headers,
                            timeout,
                            data,
                            res_data,
                            res_headers); //Send a post request

    if (ret==-1)
      {
        printf("func=%s line=%d, Failed to send a webrequest. Error = %s",__FUNCTION__,__L
        return false;
      }

 //--- Check if the post request was successful or not

    if (ret==204)
      {
        if (MQLInfoInteger(MQL_DEBUG))
          Print("Message sent to discord successfully");
      }
    else
      {
        printf("Failed to send message to discord. Json response Error = %s",CharArrayToSt
      }

 //---
    return(INIT_SUCCEEDED);
   }
```

All information and messages that you want to send to Discord must always be in a JSON format.

*Anything under the **content** key in a JSON object represents the **main text body of the message sent to Discord**.*

Running the above code sends a simple message to Discord.



Awesome! However, that was a tiresome process to handle the JSON data and everything else just to send a simple message to Discord, let's put everything in a class to make it easier to send messages without worrying about the technicalities every time.

Creating a Discord Class in MQL5

To ensure we have a standard class of functionality, let's add logging to it for tracking the errors produced by the API endpoint and the whole sending requests process.

Filename: Discord.mqh.

```
#include <errordescription.mqh>
#include <logging.mqh>
#include <jason.mqh>

class CDiscord
  {
protected:
   string m_webhook_url;
   string m_headers;
   uint m_timeout;
   CLogging logging;

   bool PostRequest(const string json);
   string GetFormattedJson(const string raw_json);

public:
                     CDiscord(const string webhook_url, const string headers="Content-Typ
                    ~CDiscord(void);

                     bool SendMessage(const string message);
                     bool SendEmbeds(const string title, const string description_, color
```

```
  };
//+------------------------------------------------------------------+
//|                                                                  |
//+------------------------------------------------------------------+
CDiscord::CDiscord(const string webhook_url, string headers="Content-Type: application/js
 m_webhook_url(webhook_url),
 m_headers(headers),
 m_timeout(timeout)
 {
//--- Initialize the logger

   logging.Config("Discord server");
   if (!logging.init())
      return;

   logging.info("Initialized",MQLInfoString(MQL_PROGRAM_NAME),__LINE__);
 }
```

We wrap the repetitive processes for sending POST requests and for converting texts to JSON format.

```
string CDiscord::GetFormattedJson(const string raw_json)
 {
   CJAVal js(NULL, jtUNDEF);
   bool b = js.Deserialize(raw_json);

   string json;
   js.Serialize(json);

   return json;
 }
//+------------------------------------------------------------------+
//|                                                                  |
//+------------------------------------------------------------------+
bool CDiscord::PostRequest(const string json)
 {
   char res[];

//--- serialize to string

   char data[];
   ArrayResize(data, StringToCharArray(json, data, 0, StringLen(json)));

//--- send data

   char res_data[];
   string res_headers=NULL;

   int ret = WebRequest("POST", m_webhook_url, m_headers, m_timeout, data, res_data, res_

   if (ret==-1)
     {
       printf("func=%s line=%d, Failed to send a webrequest. Error = %s",__FUNCTION__,__L
       return false;
     }

//--- Check if the post request was successful or not

   if (ret==204)
     {
       if (MQLInfoInteger(MQL_DEBUG))
         Print("Message sent to discord successfully");
         logging.info("Message sent to discord successfully",MQLInfoString(MQL_PROGRAM_NA
     }
   else
     {
       printf("Failed to send message to discord. Json response Error = %s",CharArrayToSt
       logging.error(CharArrayToString(res_data), MQLInfoString(MQL_PROGRAM_NAME), __LINE
     }
```

```
    return true;
  }
```

Below is a simple function for sending messages to Discord.

```
bool CDiscord::SendMessage(const string message)
 {
   string raw_json = StringFormat("{\"content\": \"%s\"}",message);
   string json = GetFormattedJson(raw_json); //Deserialize & Serialize the message in JSO

   return PostRequest(json);
 }
```

Here is how to use it.

Filename: Discord EA.mq5

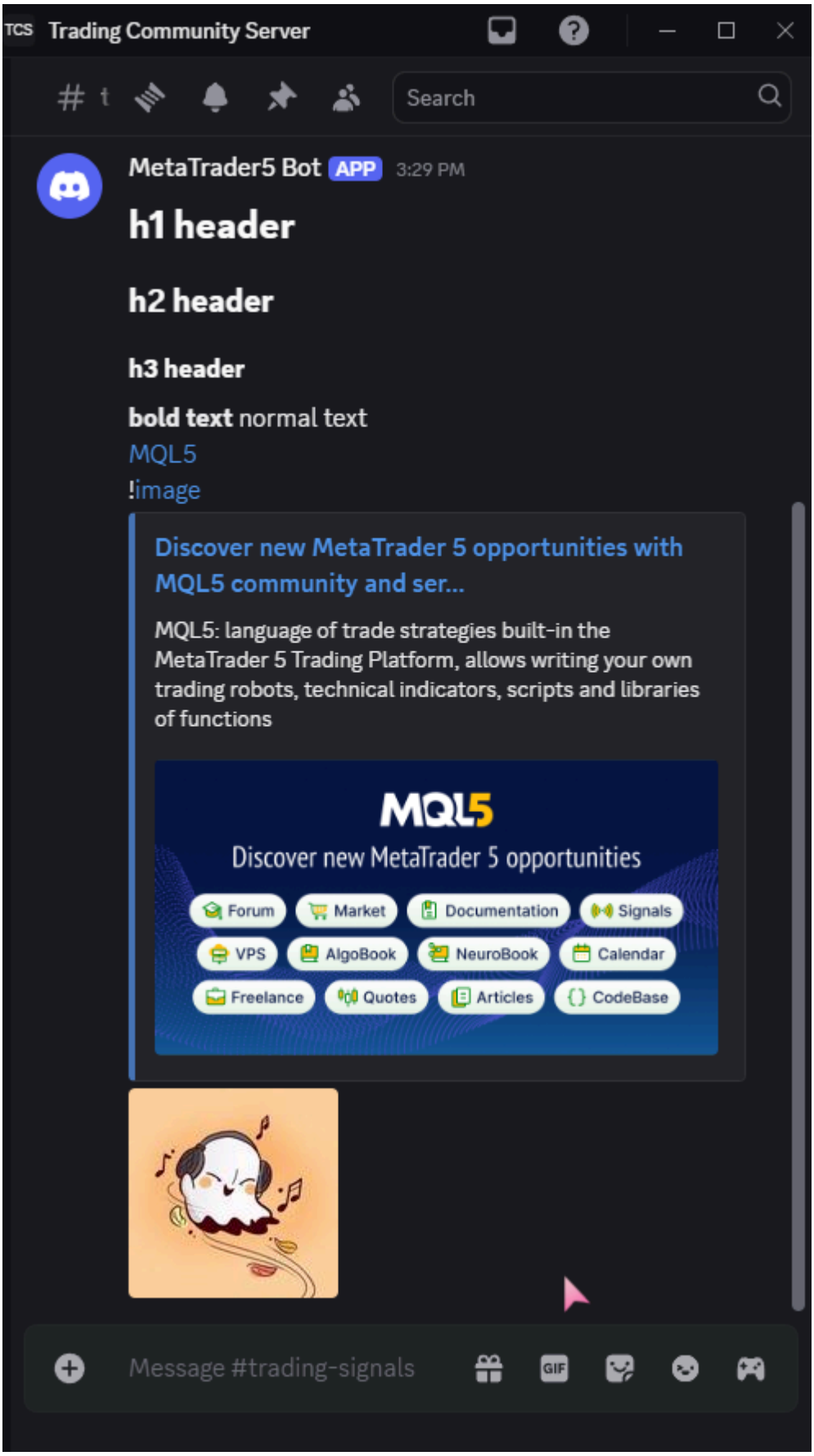
```
#include <Discord.mqh>
CDiscord *discord;

string discord_webhook_url = "https://discord.com/api/webhooks/1384809399767269527/105Kp2
//+------------------------------------------------------------------+
//| Expert initialization function                                   |
//+------------------------------------------------------------------+
int OnInit()
  {
//---

    discord = new CDiscord(discord_webhook_url);
    discord.SendMessage("Hello from MetaTrader5!");
  }
```

This simple and powerful function and can be used to send messages of different kinds, since Discord uses a Markdown formatted text.

Example Markdown message

```
    discord.SendMessage(
                        "# h1 header\n"
                        "## h2 header\n"
                        "### h3 header\n\n"
                        "**bold text** normal text\n"
                        "[MQL5](https://www.mql5.com)\n"
                        "![image](https://i.imgur.com/4M34hi2.png)"
                      );
```

Outputs.

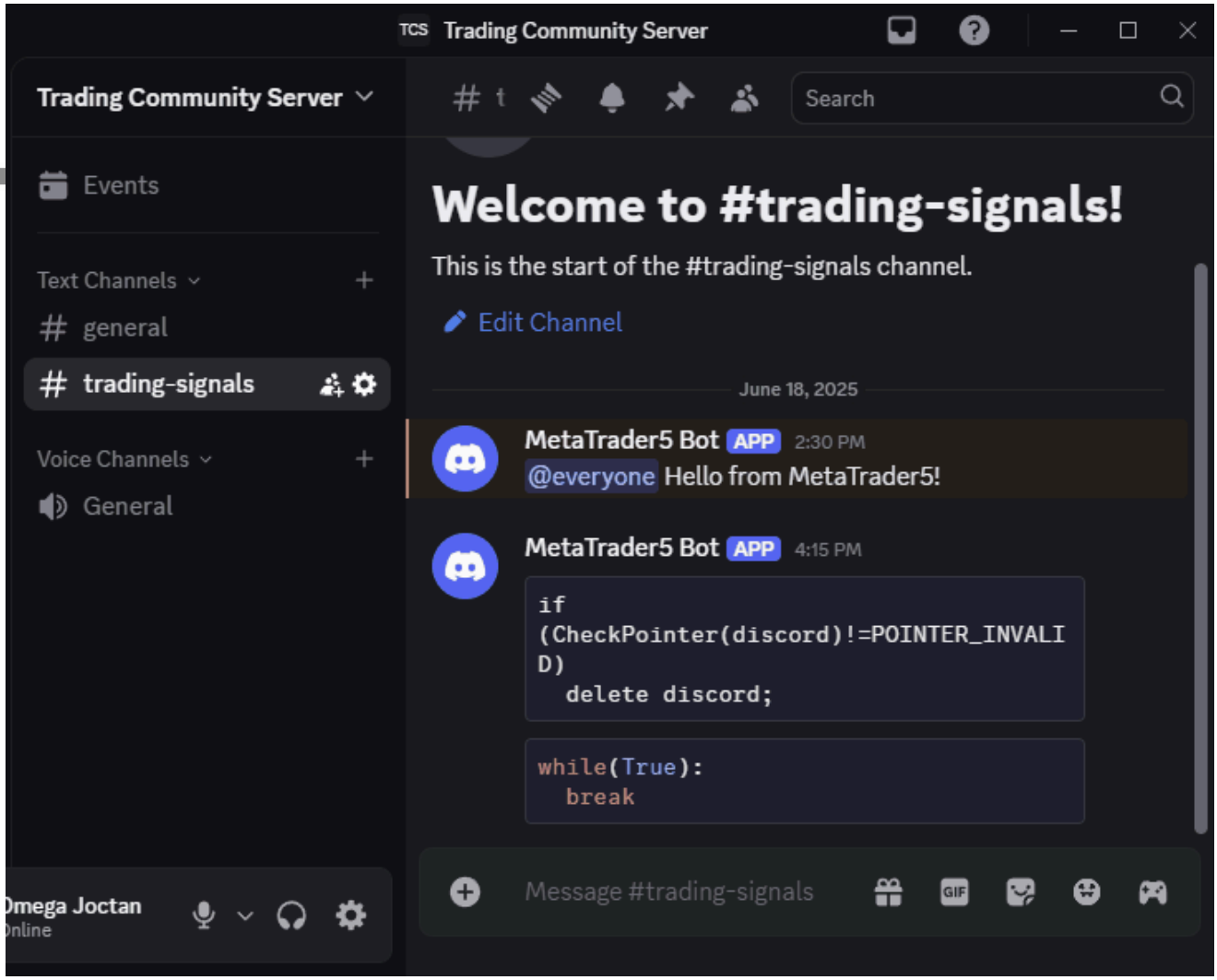


And that's not even the coolest thing you can do with the Markdown text editor provided by Discord. Let's share some lines of code with Discord.

```
      discord.SendMessage("```MQL5\n" //we put the type of language for markdown highlighte
                          "if (CheckPointer(discord)!=POINTER_INVALID)\n"
                          "  delete discord;\n"
                          "```");

      discord.SendMessage("```Python\n"
                          "while(True):\n"
```

```
                    "  break\n"
                    "```");
```

Outputs.



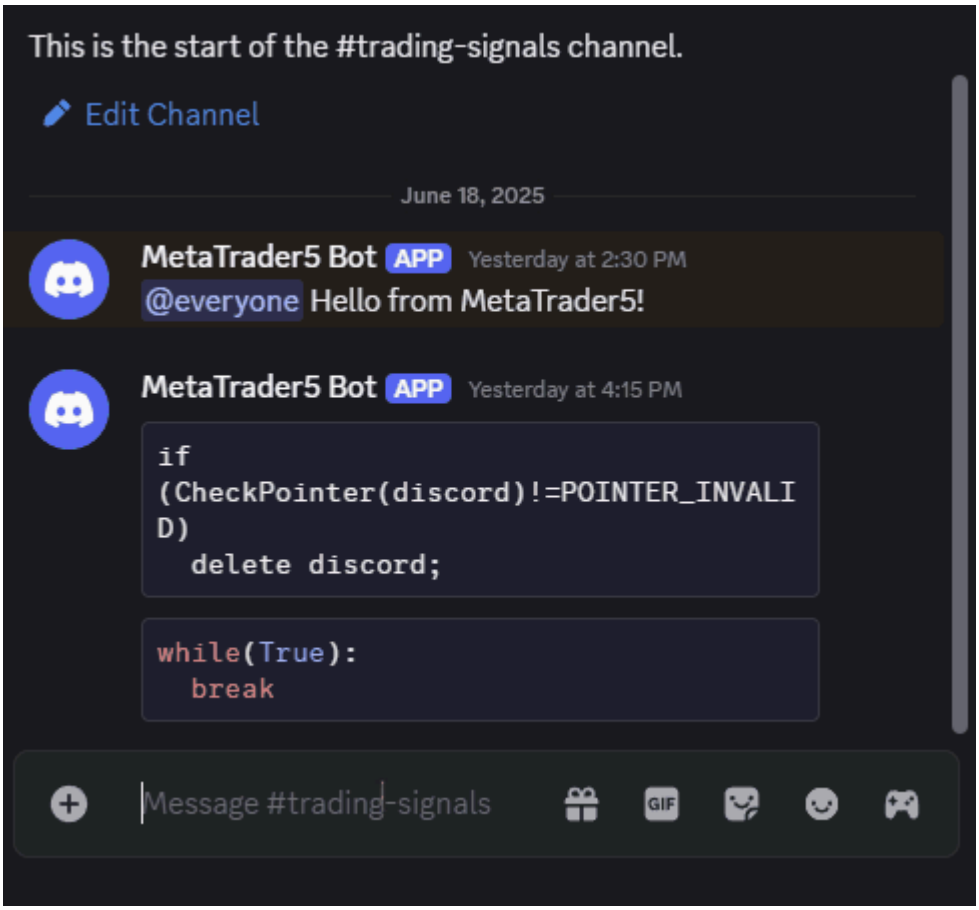
This was to show you how powerful Markdown text editors are.

Adding emoji's to your messages

Emojis are quite useful in text messages; they add elegance, improve readability, not to mention, they inject a bit of humour.

Unlike other communication platforms with emojis that are crafted from specific numeric codes, Discord uses a unique syntax to identify and render emojis directly in raw text messages.

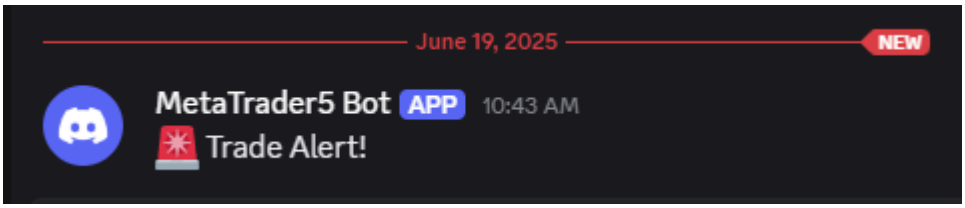
> All you have to do is to put the name of the emoji between two colons — one colon at the beginning and the other at the end of the emoji's name.



When these emojis' codes are inserted into a text message, they will be rendered as emojis in Discord.

```
discord.SendMessage(":rotating_light: Trade Alert!");
```

Message.



There are thousands of emojis, and it is challenging to keep track of them all. Here is the cheat sheet — https://github.com/ikatyang/emoji-cheat-sheet/blob/master/README.md. I have created a simple library file with some of the emoji's codes and syntax, *please don't hesitate to add more emoji's codes as it pleases you*.

Filename: discord emojis.mqh

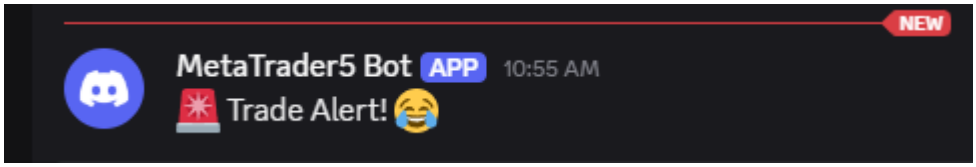
```
#define DISCORD_EMOJI_ROCKET                    ":rocket:"
#define DISCORD_EMOJI_CHART_UP                  ":chart_with_upwards_trend:"
#define DISCORD_EMOJI_CHART_DOWN                ":chart_with_downwards_trend:"
#define DISCORD_EMOJI_BAR_CHART                 ":bar_chart:"
#define DISCORD_EMOJI_BOMB                      ":bomb:"
#define DISCORD_EMOJI_THUMBS_UP                 ":thumbsup:"
#define DISCORD_EMOJI_THUMBS_DOWN               ":thumbsdown:"
#define DISCORD_EMOJI_WARNING                   ":warning:"
#define DISCORD_EMOJI_JOY                       ":joy:"
#define DISCORD_EMOJI_SOB                       ":sob:"
#define DISCORD_EMOJI_SMILE                     ":smile:"
#define DISCORD_EMOJI_FIRE                      ":fire:"
#define DISCORD_EMOJI_STAR                      ":star:"
#define DISCORD_EMOJI_BLUSH                     ":blush:"
#define DISCORD_EMOJI_THINKING                  ":thinking:"
#define DISCORD_EMOJI_ROTATING_LIGHT            ":rotating_light:"
#define DISCORD_EMOJI_X                         ":x:"
#define DISCORD_EMOJI_WHITE_CHECK_MARK          ":white_check_mark:"
#define DISCORD_EMOJI_BALLOT_BOX_WITH_CHECK     ":ballot_box_with_check:"
```

```
#define DISCORD_EMOJI_HASH                    ":hash:"
#define DISCORD_EMOJI_ASTERISK                ":asterisk:"

#define DISCORD_EMOJI_ZERO                    ":zero:"
#define DISCORD_EMOJI_ONE                     ":one:"
#define DISCORD_EMOJI_TWO                     ":two:"
#define DISCORD_EMOJI_THREE                   ":three:"
#define DISCORD_EMOJI_FOUR                    ":four:"
#define DISCORD_EMOJI_FIVE                    ":five:"
#define DISCORD_EMOJI_SIX                     ":six:"
#define DISCORD_EMOJI_SEVEN                   ":seven:"
#define DISCORD_EMOJI_EIGHT                   ":eight:"
#define DISCORD_EMOJI_NINE                    ":nine:"
#define DISCORD_EMOJI_TEN                     ":keycap_ten:"

#define DISCORD_EMOJI_RED_CIRCLE              ":red_circle:"
#define DISCORD_EMOJI_GREEN_CIRCLE            ":green_circle:"
```

Sending a message.

```
discord.SendMessage(DISCORD_EMOJI_ROTATING_LIGHT " Trade Alert! " DISCORD_EMOJI_JOY);
```

Message outcome.



Mentioning users & roles

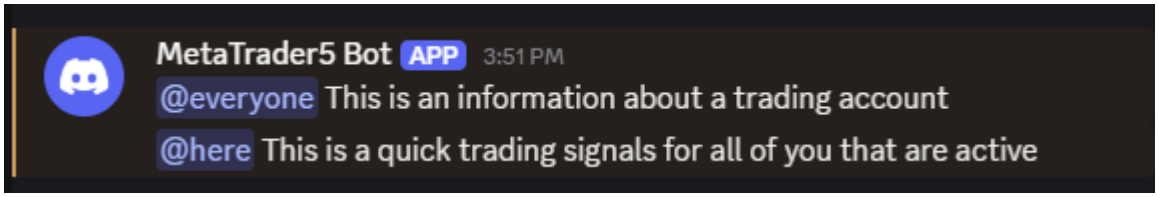
This simple message can handle mentions and roles, which are crucial for effective message delivery.

- @everyone — This notifies all users in the channel even if they are offline
- @here — This notifies all non-idle users in the chat. *All users who are online*.
- <@user_id> — This notifies a specific user or users.
- <@&role_id> — This notifies all users assigned to a specific role. For example, sending trading signals to all users who have been assigned the role of manual traders.

Example usage.

```
discord.SendMessage("@everyone This is an information about a trading account");
discord.SendMessage("@here This is a quick trading signals for all of you that are active
```

A message outcome.



Discord Bot's Identity

The good thing about a webhook is that it is not restricted to the default appearance (name and avatar); you can change these values anytime you send a post request.

This ability is quite handy as you can have multiple trading robots that send some kind of information to a community or two, all sharing one webhook, having the ability to use a different identity helps to distinguish the senders and received messages.

We can make this identity a necessity every time the discord class is called (initiated) to make the identity global.

```
class CDiscord
  {
protected:

//...
//...

public:
                      string m_name;
                      string m_avatar_url;

                      CDiscord(const string webhook_url,
                               const string name,
                               const string avatar_url,
                               const string headers="Content-Type: application/json; chars
                               const uint timeout=10000);
 }
```

We declare these variables publicly as a user might want to modify or access them along the way (after class initialization).

This makes it even easier to track the logs for each username that gets assigned to this Discord messenger.

```
CDiscord::CDiscord(const string webhook_url,
                   const string name,
                   const string avatar_url,
                   const string headers="Content-Type: application/json; charset=utf-8",
                   const uint timeout=10000):

 m_webhook_url(webhook_url),
 m_headers(headers),
 m_timeout(timeout),
 m_name(name),
 m_avatar_url(avatar_url)
 {
//--- Initialize the logger

   logging.Config(m_name);
   if (!logging.init())
      return;

   logging.info("Initialized",MQLInfoString(MQL_PROGRAM_NAME),__LINE__);
 }
```

Now, we have to append this identity (name and avatar) information everywhere where a JSON string is found.

```
bool CDiscord::SendMessage(const string message)
 {
   string raw_json = StringFormat("{"
                                    "\"username\": \"%s\","
                                    "\"avatar_url\": \"%s\","
                                    "\"content\": \"%s\""
                                  "}",
                                  m_name, m_avatar_url, message);

   string json = GetFormattedJson(raw_json); //Deserialize & Serialize the message in JSO

   return PostRequest(json);
 }
```

Inside the SendJSON() function, we need to make this identity optional because a user might have a different idea in the first place, that's why they chose to go with a function that takes a raw JSON string that gives them control

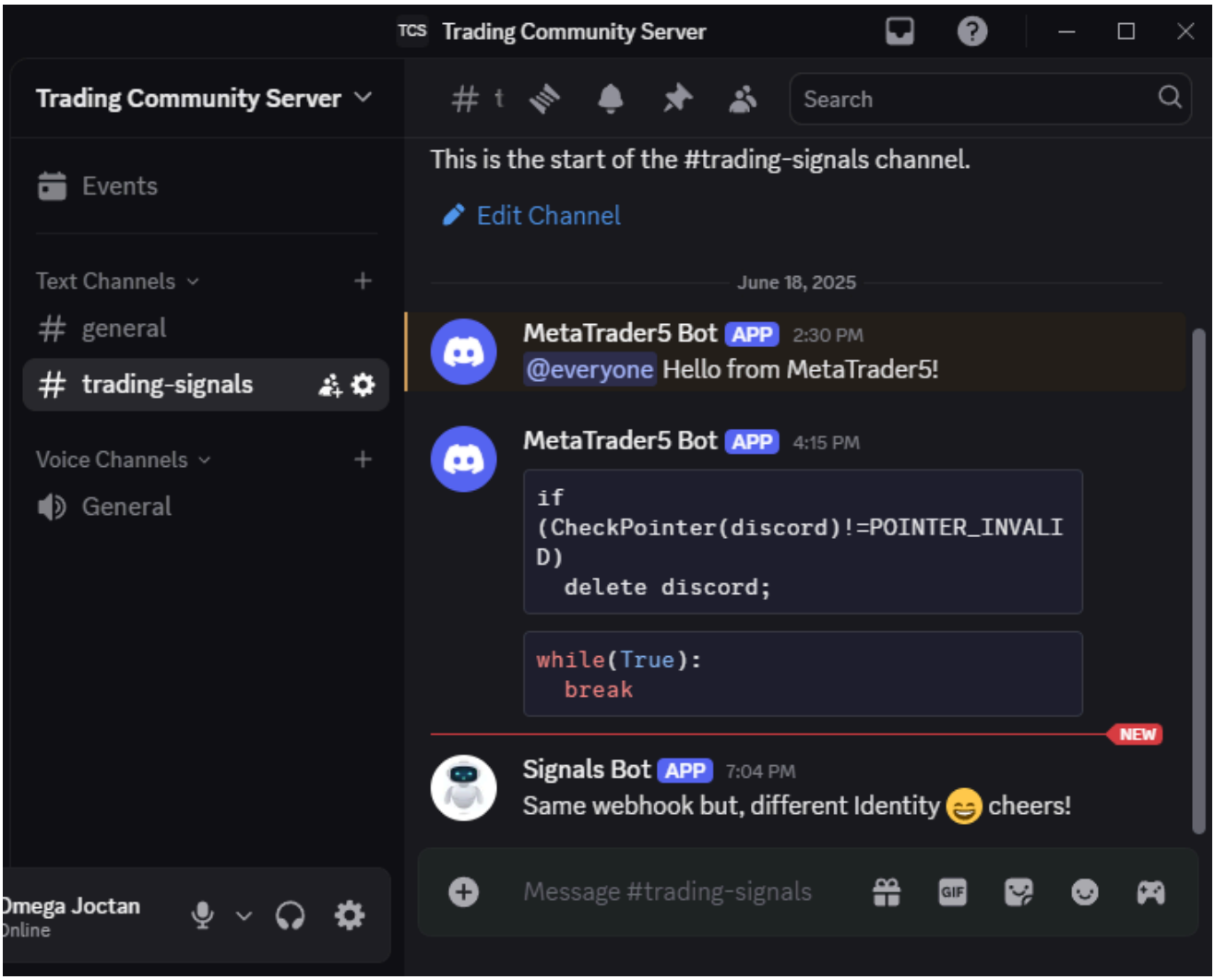
over what information they want to send.

```
bool CDiscord::SendJSON(const string raw_json, bool use_id=true)
 {
   CJAVal js;

   js.Deserialize(raw_json);

   if (use_id) //if a decides to use the ids assigned to the class constructor, we append
     {
       js["username"] = m_name;
       js["avatar_url"] = m_avatar_url;
     }

   string json;
   js.Serialize(json);

   return PostRequest(json);
 }
```

Let us set a different identity for our bot.

```
#include <Discord.mqh>
CDiscord *discord;

input string discord_webhook_url = "https://discord.com/api/webhooks/1384809399767269527/
input string avatar_url_ = "https://imgur.com/m7sVf51.jpeg";
input string bots_name = "Signals Bot";
//+------------------------------------------------------------------+
//| Expert initialization function                                   |
//+------------------------------------------------------------------+
int OnInit()
  {
//---

    discord = new CDiscord(discord_webhook_url,
                           bots_name,
                           avatar_url_);

    discord.SendMessage("Same webhook but, different Identity :smile: cheers!");

//---
   return(INIT_SUCCEEDED);
  }
```

Outcome.



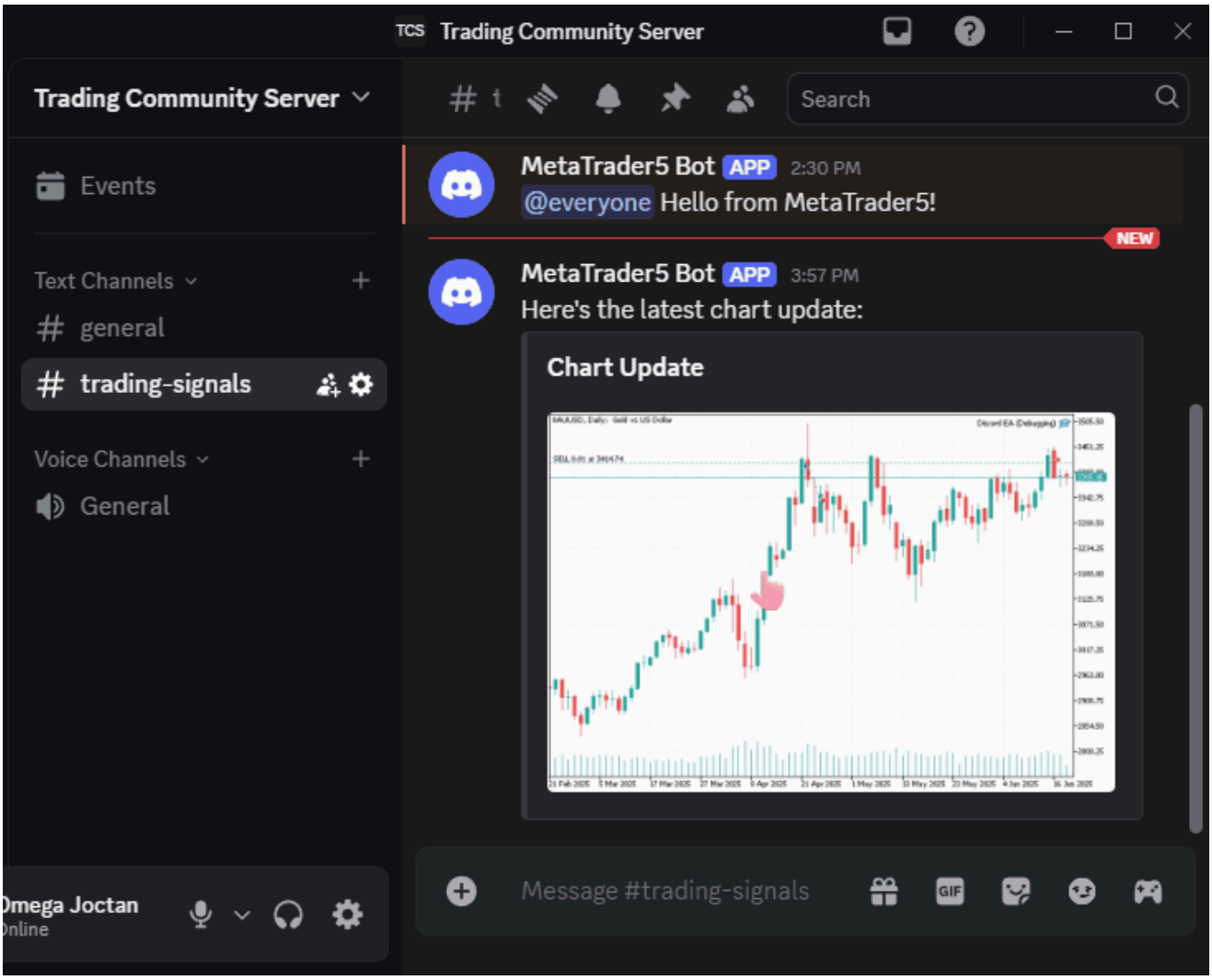
Working with Embeds and Image files

Despite having attached the image to the message discussed in the previous section when creating a Discord class, there is a proper way to attach files and embed items to a message.

We can accomplish this with an independent JSON request.

```
discord.SendJSON(
    "{"
        "\"content\": \"Here's the latest chart update:\","
        "\"embeds\": ["
            "{"
                "\"title\": \"Chart Update\","
                "\"image\": {"
                    "\"url\": \"https://imgur.com/SBsomI7.png\""
                "}"
            "}"
        "]"
    "}"
);
```

Results.



Unfortunately, you cannot send an image directly from MQL5 using Web Request, *despite Discord being capable of receiving the files*. Currently, the best you can do is to share an image from some link where the file is hosted online.

I had to use imgur.com in the previous example.

Sending Trade Notifications from MetaTrader 5 to Discord

Traders often use the ability to send information from MetaTrader 5 to external platforms to send updates about their trading activities. Let's use this bot (Expert Advisor) for such a task.

Starting with notifications for opening a trade.

01: Sending trades opening notifications

The OnTradeTransaction function is handy for this task.

Below are condition checkers for checking the state of a recent position and deal.

```
#define IS_TRANSACTION_POSITION_OPENED          (trans.type == TRADE_TRANSACTION_DEAL_ADD
#define IS_TRANSACTION_POSITION_CLOSED          (trans.type == TRADE_TRANSACTION_DEAL_ADD
#define IS_TRANSACTION_POSITION_MODIFIED        (trans.type == TRADE_TRANSACTION_POSITION)
```

```
void OnTradeTransaction(const MqlTradeTransaction& trans,
                        const MqlTradeRequest& request,
                        const MqlTradeResult& result)
  {
     ulong deal_ticket = trans.deal;
     ulong position_ticket = trans.position;

//---
```

```
      m_deal.Ticket(deal_ticket); //select a deal by it's ticket

      if (IS_TRANSACTION_POSITION_OPENED)
        {
          if (deal_ticket==0)
            return;

          if (!m_position.SelectByTicket(position_ticket)) //select a position by ticket
             return;

          if (!m_symbol.Name(m_deal.Symbol())) //select a symbol from a position
            {
             printf("line=%d Failed to select symbol %s. Error = %s",__LINE__,m_deal.Symb
             return;
            }

          string message =
             DISCORD_EMOJI_ROTATING_LIGHT " **TRADE OPENED ALERT** \n\n"
             "- **Symbol:**"+m_position.Symbol()+"\n"
             "- **Trade Type:** "+ (m_position.PositionType()==POSITION_TYPE_BUY ? DISCORD_
             "- **Entry Price:** `"+DoubleToString(m_deal.Price(), m_symbol.Digits())+"`\n
             "- **Stop Loss:** `"+DoubleToString(m_position.StopLoss(), m_symbol.Digits())
             "- **Take Profit:** `"+DoubleToString(m_position.TakeProfit(), m_symbol.Digit
             "- **Time UTC:** `"+TimeToString(TimeGMT())+"`\n"
             "- **Position Ticket:** `"+(string)position_ticket+"`\n"
             "> "DISCORD_EMOJI_WARNING" *Risk what you can afford to lose.*";

          discord.SendMessage(message);
        }

    //...
    //... Other lines of code
    //...
  }
```

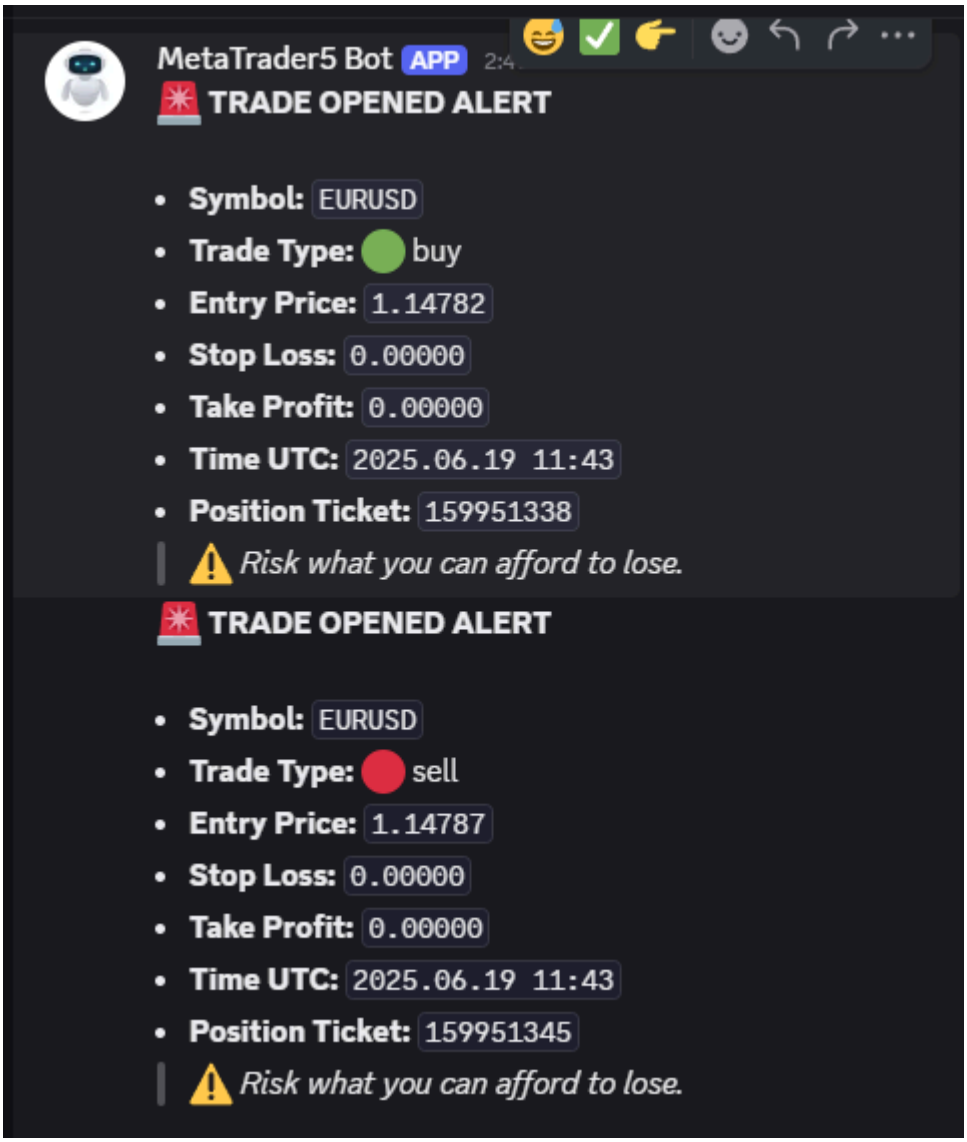
Note:

When formatting market values like entry price, stop loss, and take profit, etc., I wrap each one with a single backtick (`). This creates an inline code block in Discord, which helps visually separate the numbers and makes them easier to copy for users.

Discord uses.

- A single backtick (`) to format inline code — great for short values like prices.
- Triple backticks (```) to format multi-line code blocks — used for larger blocks of code or messages.

I opened two opposite trades manually using **One-Click Trading** in MetaTrader 5, below is the outcome.



02: Sending trades modification notifications

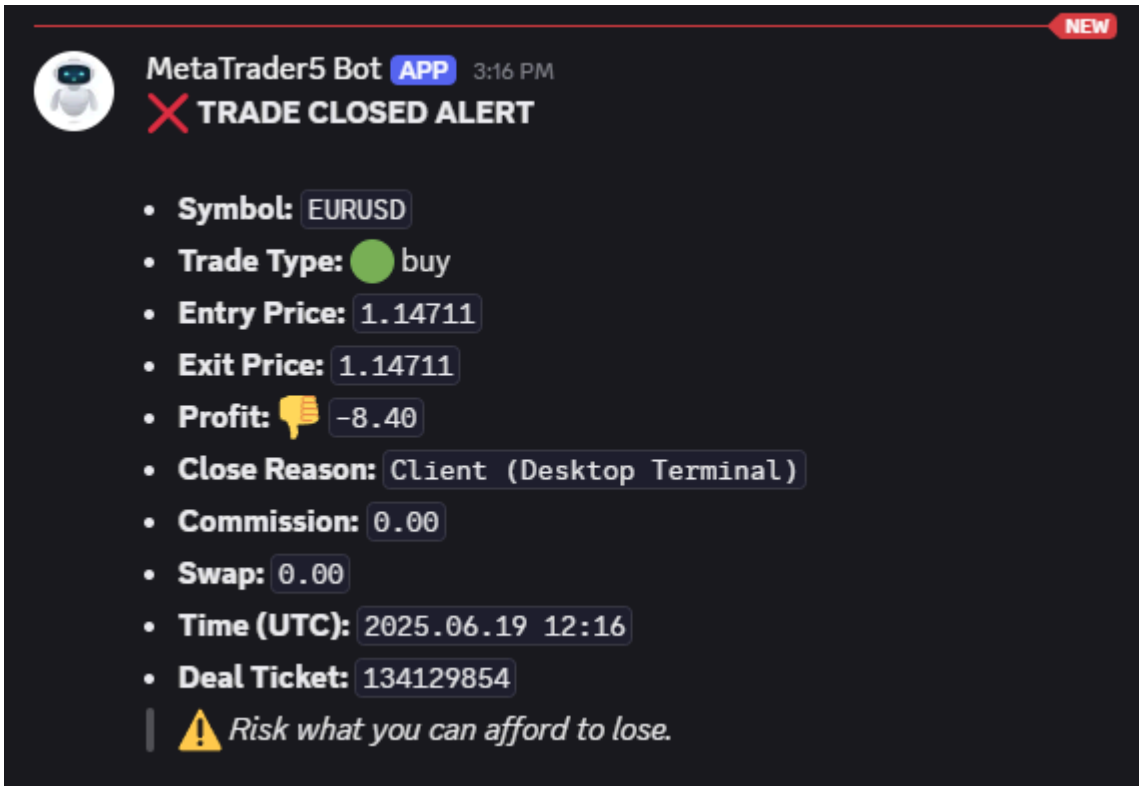
```
if (IS_TRANSACTION_POSITION_MODIFIED)
  {
    if (!m_position.SelectByTicket(position_ticket))
      {
        printf("Failed to modify a position. Erorr = %s",ErrorDescription(GetLastError())
        return;
      }

    if (!m_symbol.Name(m_deal.Symbol()))
      {
        printf("line=%d Failed to select symbol %s. Error = %s",__LINE__,m_deal.Symbol(),
        return;
      }

    string message =
       DISCORD_EMOJI_BAR_CHART " **TRADE MODIFIED ALERT** \n\n"
       "- **Symbol:** `"+m_position.Symbol()+"`\n"
       "- **Trade Type:** "+ (m_position.PositionType()==POSITION_TYPE_BUY ? DISCORD_EMOJ
       "- **Entry Price:** `"+DoubleToString(m_position.PriceOpen(), m_symbol.Digits())+"
       "- **New Stop Loss:** `"+DoubleToString(m_position.StopLoss(), m_symbol.Digits())+
       "- **New Take Profit:** `"+DoubleToString(m_position.TakeProfit(), m_symbol.Digits
       "- **Time UTC:** `"+TimeToString(TimeGMT())+"`\n"
       "- **Position Ticket:** `"+(string)position_ticket+"`\n"
       "> "DISCORD_EMOJI_WARNING" *Risk what you can afford to lose.*";

    discord.SendMessage(message);
  }
```

Everything is the same in this message as it was for sending trade notifications, only stop loss and take profit field names were changed.

03: Sending closing trades notifications

```
if (IS_TRANSACTION_POSITION_CLOSED)
   {
    if (!m_symbol.Name(m_deal.Symbol()))
      {
        printf("line=%d Failed to select symbol %s. Error = %s",__LINE__,m_deal.Symbol(),
        return;
      }

    m_symbol.RefreshRates(); //Get recent ask and bid prices

    long reason_integer;
    m_deal.InfoInteger(DEAL_REASON, reason_integer);

    string reason_text = GetDealReasonText(reason_integer);

    string message =
          DISCORD_EMOJI_X " **TRADE CLOSED ALERT**\n\n"
          "- **Symbol:** `" + m_deal.Symbol() + "`\n"
          "- **Trade Type:** " + (m_position.PositionType() == POSITION_TYPE_BUY ? DISCOR
          "- **Entry Price:** `" + DoubleToString(m_deal.Price(), m_symbol.Digits()) + "`
          "- **Exit Price:** `" + (m_position.PositionType() == POSITION_TYPE_BUY ? (Doub
          "- **Profit:** " + (m_deal.Profit() >= 0 ? DISCORD_EMOJI_THUMBS_UP : DISCORD_EM
          "- **Close Reason:** `" + reason_text+ "`\n"
          "- **Commission:** `" + DoubleToString(m_position.Commission(), 2) + "`\n"
          "- **Swap:** `" + DoubleToString(m_position.Swap(), 2)+ "`\n"
          "- **Time (UTC):** `" + TimeToString(TimeGMT()) + "`\n"
          "- **Deal Ticket:** `" + string(deal_ticket) + "`\n"
          "> "DISCORD_EMOJI_WARNING" *Risk what you can afford to lose.*";

    discord.SendMessage(message);
   }
```

On a closing trade message, we put a thumbs-up emoji when a trade was closed in profit, and a thumbs-down emoji when it was closed in a loss. Unlike opening and modifying the position, where we had a position ticket number. A closed position is no longer a position — It is a deal so, we use the deal ticket number for it.



This deal was closed manually so its reason becomes *Client (Desktop Terminal)*, this is according to ENUM_DEAL_REASON.

```
string GetDealReasonText(long reason)
{
   switch((int)reason)
   {
      case DEAL_REASON_CLIENT:            return "Client (Desktop Terminal)";
      case DEAL_REASON_MOBILE:            return "Mobile App";
      case DEAL_REASON_WEB:               return "Web Platform";
      case DEAL_REASON_EXPERT:            return "Expert Advisor";
      case DEAL_REASON_SL:                return "Stop Loss Hit";
      case DEAL_REASON_TP:                return "Take Profit Hit";
      case DEAL_REASON_SO:                return "Stop Out (Margin)";
      case DEAL_REASON_ROLLOVER:          return "Rollover Execution";
      case DEAL_REASON_VMARGIN:           return "Variation Margin Charged";
      case DEAL_REASON_SPLIT:             return "Stock Split Adjustment";
      case DEAL_REASON_CORPORATE_ACTION:  return "Corporate Action";
      default:                            return "Unknown Reason";
   }
}
```

The Bottom LIne

Discord webhooks are a good starting point for MetaTrader 5 and Discord integration, enabling communications between these two powerful platforms. However, as I said earlier, all API's have rules and limitations — Discord Webhook API is not different because Webhook requests are rate-limited.

- You can only send 5 messages per 2 every 2 seconds using a single Webhook
- A maximum of 30 messages per minute per channel is allowed for a single Webhook.

Exceeding this limitation, Discord will return HTTP 429 (Too Many Requests). To tackle this, you can add delays or queues in your MQL5 code.

Also, throughout the article, I talk about Discord bot(s), but a Webhook is very different from a Discord bot. *The entire workflow between MQL5 and Discord is what I refer to as a bot.*

> Bots are essentially programs that run within Discord, offering a wide range of functionalities, from basic commands to complex automations. Webhooks, on the other hand, are simple URLs that allow external

applications to send automated messages to a specific Discord channel

Below is the tabulated difference between a Discord Webhook and a Discord bot.

	Webhooks	Bots
Function	▪ Can only send messages to a set channel ▪ They can only send messages, not view any. ▪ Can send up to 10 embeds per message.	▪ Much more flexible as *they can do more complex actions similar to what a regular user can do*. ▪ Bots can view and send messages. ▪ Only one embed per message is allowed.
Customization	▪ Can create 10 webhooks per server with the ability to customize each avatar and name. ▪ Able to hyperlink any text outside of an embed	▪ Public bots often have a preset avatar and name, which cannot be modified by end users. ▪ Cannot hyperlink any text in a normal message, must use an embed
Load and security	▪ Just an endpoint to send data to, no actual hosting is required. ▪ No authentication that the data sent to the webhook is from a trusted source. ▪ No authentication that the data sent to the webhook is from a trusted source. If the webhook URL is leaked, only non-permanent problems may occur (e.g., spamming) ▪ Easy to change webhook URL if needed.	▪ Bots have to be hosted in a secure environment that will need to be kept online all the time, which costs more resources. ▪ Bots are authenticated via a token; a compromised token can cause severe damage due to their capabilities if they have permissions granted to them by the server owner. ▪ However, you can reset the bot token if needed.

A Discord bot is quite complex and not very practical to MQL5 programmers, as we often want a functionality to alert our fellow traders about the trading progress only. Not to mention, to create a Discord bot, you need a couple of Python modules.

However, you can develop this fully fledged Discord bot that works with MetaTrader 5 if you'd like to.

Since we have a MetaTrader 5-Python package, from the Python environment, you can work your way out to a full integration between these two platforms.

Best regards.

Attachments Table

Filename	Description & Usage
Include\discord emojis.mqh	Contain emoji codes and syntax compatible with Discord.
Include\discord.mqh	It has the CDiscord class for sending messages in JSON format to the Discord platform.
Include\errordescription.mqh	Contains descriptions of all error codes produced by MetaTrader 5 in MQL5 language.
Include\jason.mqh	A library for serialization and deserialization of JSON protocol.
Include\logging.mqh	A library for logging all the information and errors produced by the CDiscord class (Webhook sender).
Experts\Discord EA.mq5	An Expert Advisor (EA) for testing the Discord webhook and for sending trade alerts to the assigned Discord webhook URL.